

<!-- svg_diagram.md – exercises the “‘svg fenced-code passthrough.

Every fenced “‘svg block is routed through the @SVG macro (HTML mode) and inlined verbatim into the page. In PDF mode the macro is a stub: the block disappears with an apology, but the rest of the page still typesets cleanly.

The point of the example is to show what hand-drawn SVG buys you when @Mermaid is too auto-magic and @Diag is too rigid: arbitrary shapes, gradients, text-on-path, hand-tuned coordinates, and a predictable output size. ->

Hand-rolled SVG diagrams

mdlout’s @Diag package and the @Mermaid passthrough cover most day-to-day diagrams, but they are both *programs* that emit a layout. When a figure calls for hand-tuned coordinates, custom gradients, or shapes the diagram engines do not ship – a hex grid, a phase portrait, a logo, a circuit symbol – drop into a ‘svg fenced block. The body of the fence is inlined verbatim inside the page SVG via the @SVG macro.

A traffic-light state machine

Three coloured nodes, one for each light, with directed arrows labelled by their dwell times. The whole figure is 360 by 110 points, sized to sit between two paragraphs without forcing a page break:

[inline SVG omitted in non-SVG back-end]

The transitions read left-to-right (RED to AMBER to GREEN at the top) and the curved return loop closes the cycle. None of this is expressible in @Mermaid’s stateDiagram syntax without losing the fixed-pixel control over the loop curvature.

A phase portrait sketch

Pure geometry: a unit circle, two crossing axes, and a spiral trajectory drawn as a single cubic-Bezier path. mdlout’s @Diag package has no built-in spiral primitive; SVG’s <path d="..." /> syntax expresses one in a single line.

[inline SVG omitted in non-SVG back-end]

The trajectory starts at the marked point on the positive x-axis, loops inward toward the origin, and the dashed circle marks the unit-radius sphere of attraction. The viewBox is centred on the origin (negative-half to positive-half on each axis) so the SVG coordinate system matches the math.

A custom logo / mark

When a document needs a one-off mark – a chapter ornament, a publisher’s colophon – inline SVG is the lightest path. No external asset to ship, no font to embed:

[inline SVG omitted in non-SVG back-end]

The gradient `<linearGradient>` is what makes this hard to duplicate in PostScript-only Lout: PostScript supports gradients, but `z49.c` does not expose the syntax in a way that hand-rolled content can drive. In the SVG back-end the gradient is just SVG.

A coordinate grid for a manual layout

Sometimes you want a *blueprint* feel – gridded background, labelled axes, hand-placed shapes. Below is an architectural-style floor plan, with grid lines, room outlines, and a door swing:

[inline SVG omitted in non-SVG back-end]

The `<pattern>` element gives the grid background; the door-swing arc is a single arc-to (A) segment. None of this would be practical to draw in `@Diag` – the package is good at boxes-and- arrows, not at architectural layout.

When to reach for @SVG

A short decision rule:

- *Boxes-and-arrows*: stay in `@Diag` or `@Mermaid`. The auto-layout is faster than hand-placing nodes.
- *Mathematical/scientific figures*: `@Diag` if the figure is trees / graphs; raw SVG if it is geometry (phase portraits, contour plots, vector fields).
- *Logos, ornaments, blueprint backdrops*: raw SVG every time. These need pixel-level control and the diagram engines do not give it.
- *Charts*: usually pre-render in `matplotlib` / `D3` / `observable` and `![] (chart.svg)` the result. Hand-rolling a chart in inline SVG is possible but rarely worth the labour.

The PDF route is the catch: every `@SVG` block in PDF mode falls back to a placeholder. If the figure must appear in the print build, render it once with `-format=html, screenshot or pdftocairo -svg` the result, save the file alongside the source, and reference it as `![] (figure.svg)` – which the SVG back-end inlines and the PostScript back-end ships through ImageMagick.